

Report: Traffic Collision Data Analysis

1. Data Preparation

1.1. Loading the dataset

```
[4]: # Write code to Load the data and check the schema
case = spark.read.option('header',True).csv("sample_case_ids.csv")
collision = spark.read.option('header',True).csv("sample_collisions.csv")
parties = spark.read.option('header',True).csv("sample_parties.csv")
victims = spark.read.option('header',True).csv("sample_victims.csv")

print('Case dataset schema :', case.schema)
print('Collision dataset schema :', collision.schema)
print('Parties dataset schema :', parties.schema)
print('Victims dataset schema :', victims.schema)

Case dataset schema : StructType([StructField('case_id', StringType(), True), StructField('db_year', StringType(), True)])
Collision dataset schema : StructType([StructField('case_id', StringType(), True), StructField('jurisdiction', StringType(), True), StructField('officer_id', StringType(), True), StructField('reporting_district', StringType(), True), StructField('chp_shift', StringType(), True), StructField('population', StringType(), True), StructField('county_city_location', StringType(), True), StructField('county_location', StringType(), True), StructField('special_condition', StringType(), True), StructField('beat_type', StringType(), True), StructField('beat_type', StringType(), True), StructField('chp_beat_class', StringType(), True), StructField('beat_number', StringType(), True), StructField('primary_road', StringType(), True), StructField('secondary_road', StringType(), True), StructField('distance', StringType(), True), StructField('direction', StringType(), True), StructField('intersection', StringType(), True), StructField('weather_1', StringType(), True), StructField('state_highway_indicator', StringType(), True), StructField('caltrans_county', StringType(), True), StructField('caltrans_district', StringType(), True), StructField('state_route', StringType(), True), StructField('postmile', StringType(), True), StructField('location_type', StringType(), True), StructField('side_of_highway', StringType(), True), StructField('tow_away', StringType(), True), StructField('collision_severity', StringType(), True), StructField('killed_victims', StringType(), True), StructField('injured_victims', StringType(), True), StructField('party_count', StringType(), True), StructField('primary_collision_factor', StringType(), True), StructField('pcf_violation_category', StringType(), True), StructField('pcf_violation', StringType(), True), StructField('pcf_violation_subsector', StringType(), True)])
```

2. Data Cleaning

Before cleaning the datasets:

```
print('Number of columns in collision dataset is ',len(collision.columns))
print('Number of columns in case dataset is ',len(case.columns))
print('Number of columns in parties dataset is ',len(parties.columns))
print('Number of columns in victims dataset is ',len(victims.columns))
```

```
Number of columns in collision dataset is 65
Number of columns in case dataset is 2
Number of columns in parties dataset is 24
Number of columns in victims dataset is 11
```

```
] print('Number of rows in collision dataset is ',collision.count())
print('Number of rows in case dataset is ',case.count())
print('Number of rows in parties dataset is ',parties.count())
print('Number of rows in victims dataset is ',victims.count())
```

```
Number of rows in collision dataset is 935791
Number of rows in case dataset is 942433
Number of rows in parties dataset is 1866917
Number of rows in victims dataset is 963933
```

2.1. Missing Values

Null count of collision dataset columns:

```

get_null_count(collision)

jurisdiction            0
officer_id              1158
reporting_district      2264
population              552653
special_condition       165
chp_beat_class          51730
beat_number             274
primary_road            80780
secondary_road          10
direction               2
intersection            232663
weather_1               9484
state_highway_indicator 4626
caltrans_county         318
caltrans_district       682075
state_route             648728
postmile                648728
location_type           648728
side_of_highway         682077
tow_away                6153
killed_victims          200
injured_victims         230
primary_collision_factor 5660
pcf_violation_category  15424
pcf_violation           64175
pcf_violation_subsection 601427
type_of_collision       7655
motor_vehicle_involved_with 4887
pedestrian_action       615
road_surface            8141
road_condition_1        7903
lighting                5301
control_device          5791
statewide_vehicle_type_at_fault 186119
chp_vehicle_type_at_fault 251662
latitude                669049
longitude                669049
collision_time          8084

```

Null count of case, parties, victims dataset columns:

```
[18]: get_null_count(case)

Empty DataFrame
Columns: [0]
Index: []

[19]: get_null_count(parties)

      0
party_type      4395
party_sex      249269
party_age      290783
party_sobriety    55663
direction_of_travel    52377
party_safety_equipment_1    314883
party_safety_equipment_2    699581
financial_responsibility    152613
cellphone_in_use    568197
cellphone_use_type    432264
other_associate_factor_1    112496
movement_preceding_collision    33275
vehicle_year    177599
vehicle_make    195057
statewide_vehicle_type    210693
chp_vehicle_type_towing    401142
chp_vehicle_type_towed    909543
party_race    436844

[20]: get_null_count(victims)

      0
victim_role      1
victim_sex      24041
victim_age      32145
victim_seating_position    1913
victim_safety_equipment_1    55215
victim_safety_equipment_2    310220
victim_ejected    4272
```

Case dataset has no null values.

#Drop Sparse Columns

Below is the list of columns which has lots of null values. Hence, dropping those columns

```
#Drop Sparse Columns

parties_sparse_columns = ['party_safety_equipment_2', 'chp_vehicle_type_towed', 'party_race', 'chp_vehicle_type_towing', \
                          'cellphone_in_use', 'cellphone_use_type']
victims_sparse_columns = ['victim_safety_equipment_2']
collision_sparse_columns = ['latitude', 'longitude', 'pcf_violation_subsection', 'caltrans_county', 'caltrans_district', 'state_route', \
                           'postmile', 'location_type', 'side_of_highway', 'reporting_district', 'chp_vehicle_type_at_fault', 'direction', \
                           'statewide_vehicle_type_at_fault', 'beat_number']

parties = parties.drop(*parties_sparse_columns)
victims = victims.drop(*victims_sparse_columns)
collision = collision.drop(*collision_sparse_columns)
```

#Convert Data Types

We have numeric columns, date columns and time columns. All are in String as of now. So I am converting those into its respective datatypes.

```

: #Convert Data Types

from pyspark.sql.types import IntegerType, DateType, TimestampType

parties_numerical_columns = ['party_number', 'at_fault', 'party_age', 'party_number_killed', \
                             'party_number_injured', 'vehicle_year']
victims_numerical_columns = ['party_number', 'victim_age']
collision_numerical_columns = ['special_condition', 'distance', 'intersection', 'state_highway_indicator', 'tow_away', 'killed_victims', \
                              'injured_victims', 'party_count', 'chp_road_type', 'pedestrian_collision', 'bicycle_collision', 'motorcycle_collision', \
                              'truck_collision', 'not_private_property', 'severe_injury_count', 'other_visible_injury_count', \
                              'complaint_of_pain_injury_count', 'pedestrian_killed_count', 'pedestrian_injured_count', \
                              'bicyclist_killed_count', 'bicyclist_injured_count', 'motorcyclist_killed_count', 'motorcyclist_injured_count']

collision_date_columns = ['collision_date', 'process_date']
collision_time_columns = ['collision_time']

case_numerical_columns = ['db_year']

for column in parties_numerical_columns:
    parties = parties.withColumn(column, col(column).cast(IntegerType()))

for column in victims_numerical_columns:
    victims = victims.withColumn(column, col(column).cast(IntegerType()))

for column in collision_numerical_columns:
    collision = collision.withColumn(column, col(column).cast(IntegerType()))

for column in collision_date_columns:
    collision = collision.withColumn(column, col(column).cast(DateType()))

for column in collision_time_columns:
    collision = collision.withColumn(column, col(column).cast(TimestampType()))

```

#Handle Missing Values

Handling null values in Collision dataset

Since the highlighted columns has very fewer null values, I am dropping those rows.

```

#Handle Missing Values

# Handling null values in Collision dataset
get_null_count(collision)

```

	0
jurisdiction	1158
officer_id	2264
population	165
special_condition	51730
chp_beat_class	274
primary_road	10
secondary_road	2
intersection	9484
weather_1	4626
state_highway_indicator	318
tow_away	6153
killed_victims	200
injured_victims	230
primary_collision_factor	5660
pcf_violation_category	15424
pcf_violation	64175
type_of_collision	7655
motor_vehicle_involved_with	4887
pedestrian_action	615
road_surface	8141
road_condition_1	7903
lighting	5301
control_device	5791
collision_time	8084

```

collision = collision.na.drop(subset=["primary_road", "secondary_road", "state_highway_indicator", "killed_victims", "injured_victims", \
                                     "pedestrian_action", "collision_time", "jurisdiction", "officer_id", "population", "chp_beat_class"])
get_null_count(collision)

```

For remaining columns, I am filling null with mode value since those columns are categorical.

```
collision = collision.na.drop(subset=["primary_road", "secondary_road", "state_highway_indicator", "killed_victims", "injured_victims", \
    "pedestrian_action", "collision_time", "jurisdiction", "officer_id", "population", "chp_beat_class"])
get_null_count(collision)
```

```

0
special_condition      51246
intersection           9245
weather_1              4372
tow_away               5899
primary_collision_factor 5413
pcf_violation_category 15001
pcf_violation          62346
type_of_collision       7428
motor_vehicle_involved_with 4723
road_surface            7779
road_condition_1        7630
lighting                4848
control_device          5568
```

```
def get_mode_value(df, column):
    mode = df.groupby(column).count().orderBy(desc('count')).first()[0]
    return mode
```

```
collision = collision.na.fill({"special_condition": get_mode_value(collision, 'special_condition')})
collision = collision.na.fill({'intersection': get_mode_value(collision, 'intersection')})
collision = collision.na.fill({'weather_1': get_mode_value(collision, 'weather_1')})
collision = collision.na.fill({'primary_collision_factor': get_mode_value(collision, 'primary_collision_factor')})
collision = collision.na.fill({'tow_away': get_mode_value(collision, 'tow_away')})
collision = collision.na.fill({'pcf_violation_category': get_mode_value(collision, 'pcf_violation_category')})
collision = collision.na.fill({'pcf_violation': get_mode_value(collision, 'pcf_violation')})
collision = collision.na.fill({'type_of_collision': get_mode_value(collision, 'type_of_collision')})
collision = collision.na.fill({'motor_vehicle_involved_with': get_mode_value(collision, 'motor_vehicle_involved_with')})
collision = collision.na.fill({'road_surface': get_mode_value(collision, 'road_surface')})
collision = collision.na.fill({'road_condition_1': get_mode_value(collision, 'road_condition_1')})
collision = collision.na.fill({'control_device': get_mode_value(collision, 'control_device')})
collision = collision.na.fill({'lighting': get_mode_value(collision, 'lighting')})
```

All the null values are handled in collision dataset.

Handling null values in Victims dataset

Since the highlighted columns has very fewer null values, I am dropping those rows.

And for 'victim_sex' and 'victim_safety_equipment_1' columns, I am filling null with mode value

And for 'victim_age' column, I am filling null values with median value.

```
[30]: # Handling null values in Victims dataset
get_null_count(victims)
```

```

0
victim_role          1
victim_sex           24041
victim_age           32145
victim_seating_position 1913
victim_safety_equipment_1 55215
victim_ejected       4272
```

```
[31]: victims = victims.na.drop(subset=['victim_role', 'victim_seating_position', 'victim_ejected'])
```

```
[32]: victims = victims.na.fill({'victim_sex': get_mode_value(victims, 'victim_sex')})
victims = victims.na.fill({'victim_safety_equipment_1': get_mode_value(victims, 'victim_safety_equipment_1')})

# Median of age
median_age = int(victims.approxQuantile("victim_age", [0.5], 0.01)[0])
victims = victims.na.fill({'victim_age': median_age})
```

```
[33]: get_null_count(victims)
```

```

Empty DataFrame
Columns: [0]
Index: []
```

All the null values are handled in Victims dataset.

Handling null values in Victims dataset

By following the similar steps, I have handled all nulls in Parties dataset.

```
[34]: # Handling null values in parties dataset
      get_null_count(parties)

      0
party_type      4395
party_sex      249269
party_age      290783
party_sobriety  55663
direction_of_travel  52377
party_safety_equipment_1  314883
financial_responsibility  152613
other_associate_factor_1  112496
movement_preceding_collision  33275
vehicle_year    177599
vehicle_make    195057
statewide_vehicle_type  210693

[35]: parties = parties.drop(subset = ['party_type', 'movement_preceding_collision'])

parties = parties.na.fill({'party_sex':get_mode_value(parties,'party_sex')})
parties = parties.na.fill({'party_sobriety':get_mode_value(parties,'party_sobriety')})
parties = parties.na.fill({'direction_of_travel':get_mode_value(parties,'direction_of_travel')})
parties = parties.na.fill({'party_safety_equipment_1':get_mode_value(parties,'party_safety_equipment_1')})
parties = parties.na.fill({'financial_responsibility':get_mode_value(parties,'financial_responsibility')})
parties = parties.na.fill({'vehicle_make':get_mode_value(parties,'vehicle_make')})
parties = parties.na.fill({'statewide_vehicle_type':get_mode_value(parties,'statewide_vehicle_type')})

parties = parties.drop('other_associate_factor_1')

median_year = int(parties.approxQuantile("vehicle_year", [0.5], 0.01)[0])
parties = parties.na.fill({'vehicle_year':median_year})

median_age = int(parties.approxQuantile("party_age", [0.5], 0.01)[0])
parties = parties.na.fill({'party_age':median_age})

[36]: get_null_count(parties)

Empty DataFrame
Columns: [0]
Index: []
```

2.2. Fixing columns

Removed Duplicates

```
Number of duplicate rows removed from collision is 0
Number of duplicate rows removed from case is 35
Number of duplicate rows removed from victims is 0
Number of duplicate rows removed from parties is 0
```

Upper and Lower bounds for analysing outliers

```
Parties bound: {'party_number': {'lower_bound': -0.5, 'upper_bound': 3.5}, 'party_age': {'lower_bound': -3.0, 'upper_bound': 77.0}, 'party_number_killed': {'lower_bound': 0.0, 'upper_bound': 0.0}, 'party_number_injured': {'lower_bound': 0.0, 'upper_bound': 0.0}}
Victims bound: {'party_number': {'lower_bound': -0.5, 'upper_bound': 3.5}, 'victim_age': {'lower_bound': -19.5, 'upper_bound': 80.5}}
Collision bound: {'distance': {'lower_bound': -675.0, 'upper_bound': 1125.0}, 'state_highway_indicator': {'lower_bound': -1.5, 'upper_bound': 2.5}, 'killed_victims': {'lower_bound': 0.0, 'upper_bound': 0.0}, 'injured_victims': {'lower_bound': -1.5, 'upper_bound': 2.5}, 'party_count': {'lower_bound': 2.0, 'upper_bound': 2.0}, 'severe_injury_count': {'lower_bound': 0.0, 'upper_bound': 0.0}, 'other_visible_injury_count': {'lower_bound': 0.0, 'upper_bound': 0.0}, 'complaint_of_pain_injury_count': {'lower_bound': -1.5, 'upper_bound': 2.5}, 'pedestrian_killed_count': {'lower_bound': 0.0, 'upper_bound': 0.0}, 'pedestrian_injured_count': {'lower_bound': 0.0, 'upper_bound': 0.0}, 'bicyclist_killed_count': {'lower_bound': 0.0, 'upper_bound': 0.0}, 'bicyclist_injured_count': {'lower_bound': 0.0, 'upper_bound': 0.0}, 'motorcyclist_killed_count': {'lower_bound': 0.0, 'upper_bound': 0.0}, 'motorcyclist_injured_count': {'lower_bound': 0.0, 'upper_bound': 0.0}}
```

2.3. Outlier Analysis

Removed outliers from parties dataset

```

I]: #Remove Outliers

# Removing outliers in parties dataset
parties_numerical_columns = ['party_number', 'party_age', 'party_number_killed'] # removable outliers column
parties_cleaned = parties
for column in parties_numerical_columns:
    parties_cleaned = parties_cleaned.filter((col(column) >= parties_lb_ub_dict[column]['lower_bound']) & (col(column) <= parties_lb_ub_dict[column]['upper_bound']))
    print(column, parties_cleaned.count())

# Median using approxQuantile
median_value = parties_cleaned.approxQuantile("party_number_injured", [0.5], 0.01)[0]

parties_cleaned = parties_cleaned.withColumn(
    "party_number_injured",
    when((col("party_number_injured") < parties_lb_ub_dict[column]['lower_bound']) | (col("party_number_injured") > \
        parties_lb_ub_dict[column]['upper_bound'])), median_value)
    .otherwise(col("party_number_injured"))
)

party_number 1796801
party_age 1768203
party_number_killed 1762673

I]: parties_cleaned.count()
I]: 1762673

```

We don't have any outliers in victims dataset

3. Exploratory Data Analysis

3.1. Data Preparation

Classified the columns into Numerical and categorical

```

: # Categorical variables

parties_categorical_columns = ['party_type', 'party_sex', 'party_sobriety', 'direction_of_travel',
    'financial_responsibility', 'movement_preceding_collision', 'statewide_vehicle_type']
victims_categorical_columns = ['victim_role', 'victim_sex', 'victim_degree_of_injury', 'victim_seating_position', \
    'victim_ejected']
collision_categorical_columns = ['chp_shift', 'population', 'beat_type', 'chp_beat_type', 'chp_beat_class', 'weather_1', \
    'collision_severity', 'primary_collision_factor', 'pcf_violation_category', 'hit_and_run', \
    'type_of_collision', 'road_surface', 'road_condition_1', 'lighting']

# Numerical variables

parties_numerical_columns = ['party_number', 'at_fault', 'party_age', 'party_number_killed', \
    'party_number_injured', 'vehicle_year']
victims_numerical_columns = ['party_number', 'victim_age']
collision_numerical_columns = ['special_condition', 'distance', 'intersection', 'state_highway_indicator', 'tow_away', \
    'killed_victims', 'injured_victims', 'party_count', 'chp_road_type', 'pedestrian_collision', \
    'bicycle_collision', 'motorcycle_collision', 'truck_collision', 'not_private_property', \
    'severe_injury_count', 'other_visible_injury_count', 'complaint_of_pain_injury_count', \
    'pedestrian_killed_count', 'pedestrian_injured_count', 'bicyclist_killed_count', \
    'bicyclist_injured_count', 'motorcyclist_killed_count', 'motorcyclist_injured_count']

```

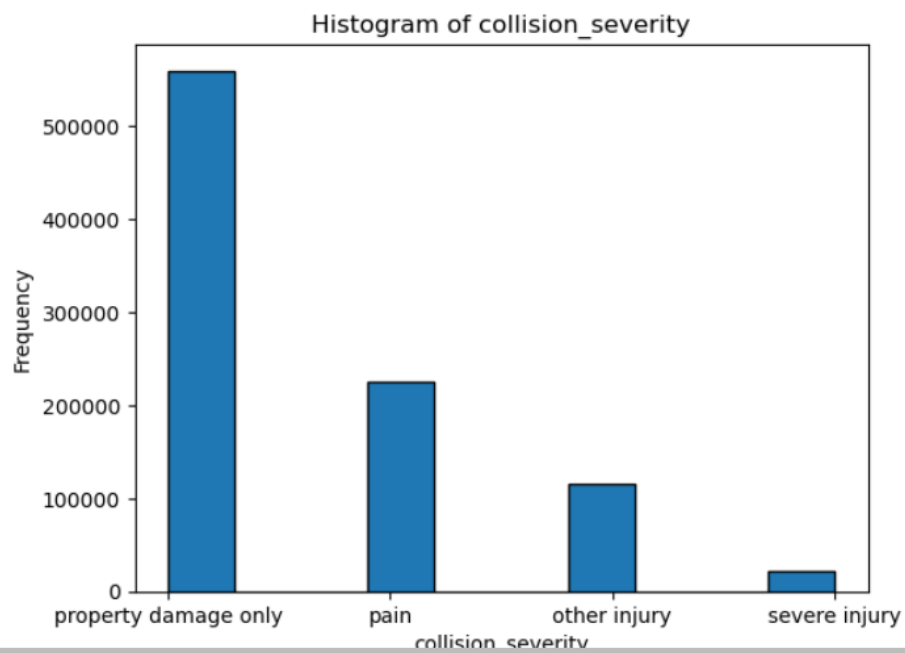
3.2. Analyze the distribution of collision severity.

Property damage is the most collision severity

```
# Convert to Pandas
pdf = collision.select("collision_severity").toPandas()

# Plot

plt.hist(pdf["collision_severity"], bins=10, edgecolor="black")
plt.xlabel("collision_severity")
plt.ylabel("Frequency")
plt.title("Histogram of collision_severity")
plt.show()
```

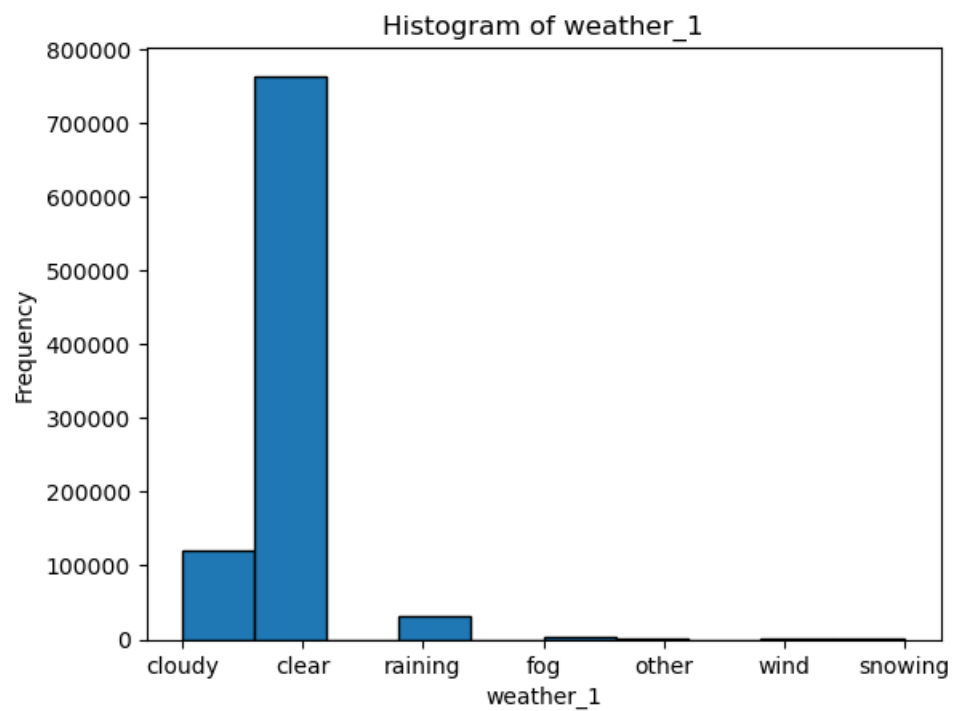


Most of the collision happened when weather is clear and during daylight


```
# Convert to Pandas

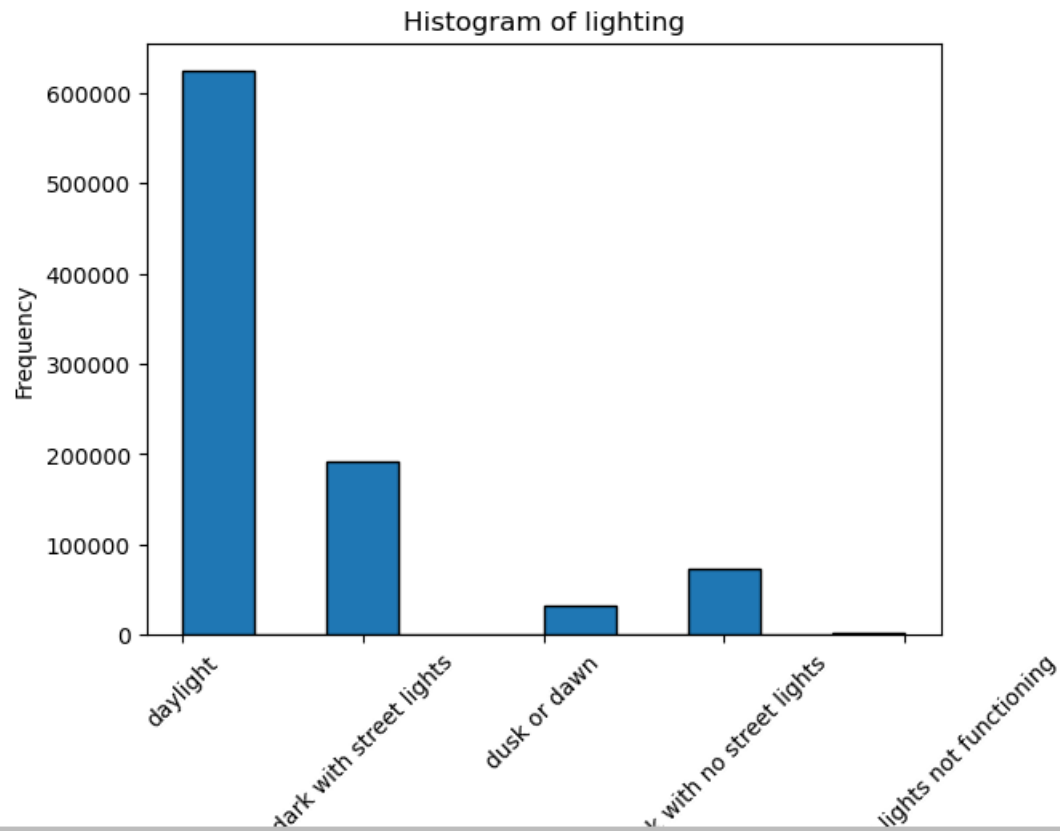
# Plot
pdf = collision.select("weather").toPandas()

# Plot histogram
plt.hist(pdf["weather"], bins=10, edgecolor="black")
plt.xlabel("weather_1")
plt.ylabel("Frequency")
plt.title("Histogram of weather_1")
plt.show()
```



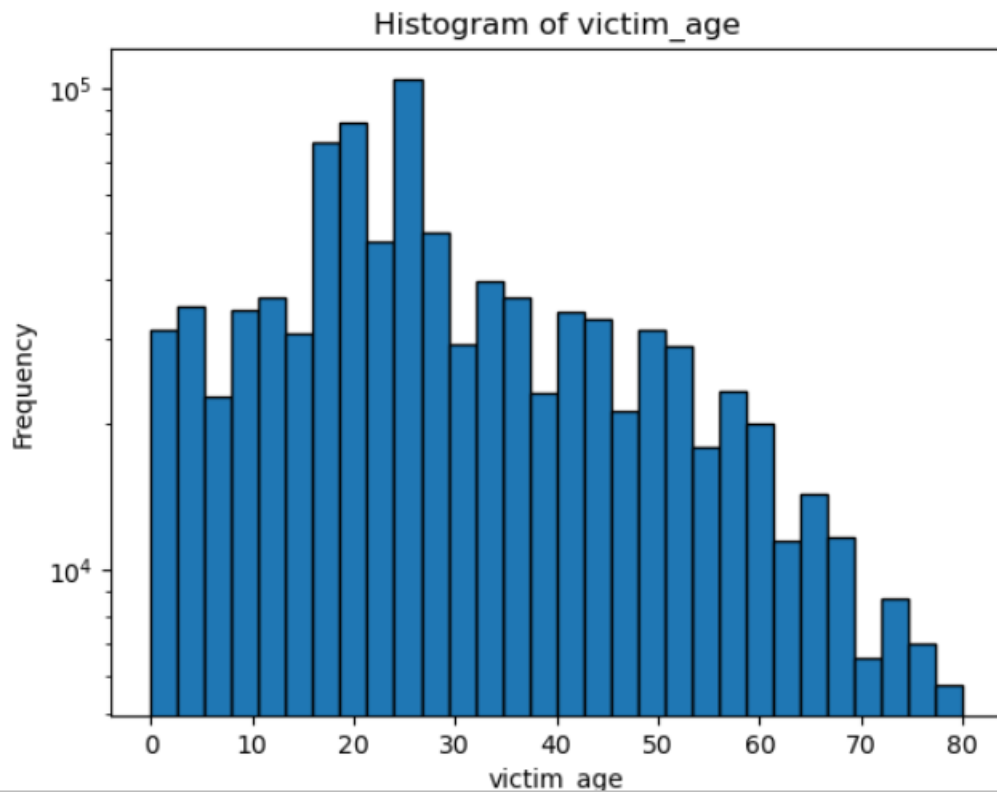
```
: pdf = collision.select("lighting").toPandas()

# Plot histogram
plt.hist(pdf["lighting"], edgecolor="black")
plt.xlabel("lighting")
plt.ylabel("Frequency")
plt.title("Histogram of lighting")
plt.xticks(rotation=45)
plt.show()
```



```
# Plot
pdf = victims.select("victim_age").toPandas()

# Plot histogram
plt.hist(pdf["victim_age"], bins=30, edgecolor="black")
plt.xlabel("victim_age")
plt.ylabel("Frequency")
plt.title("Histogram of victim_age")
plt.yscale("log")
plt.show()
```

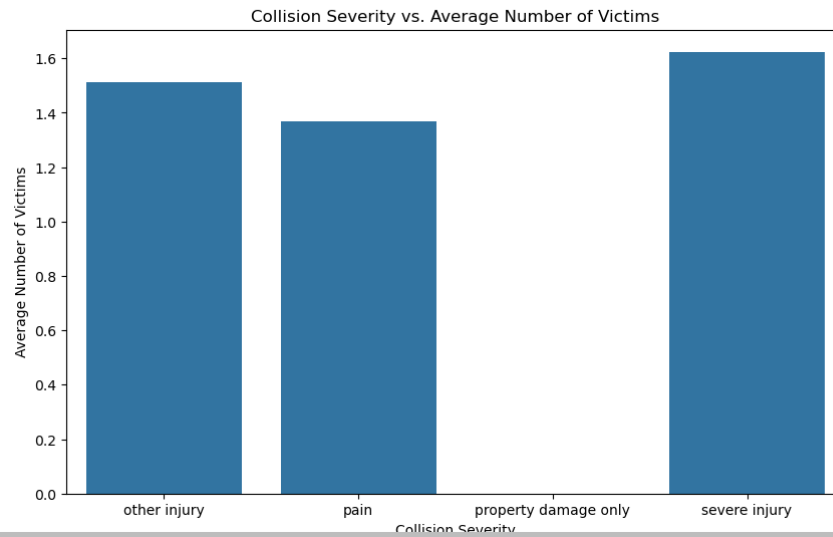


Bivariate analysis:

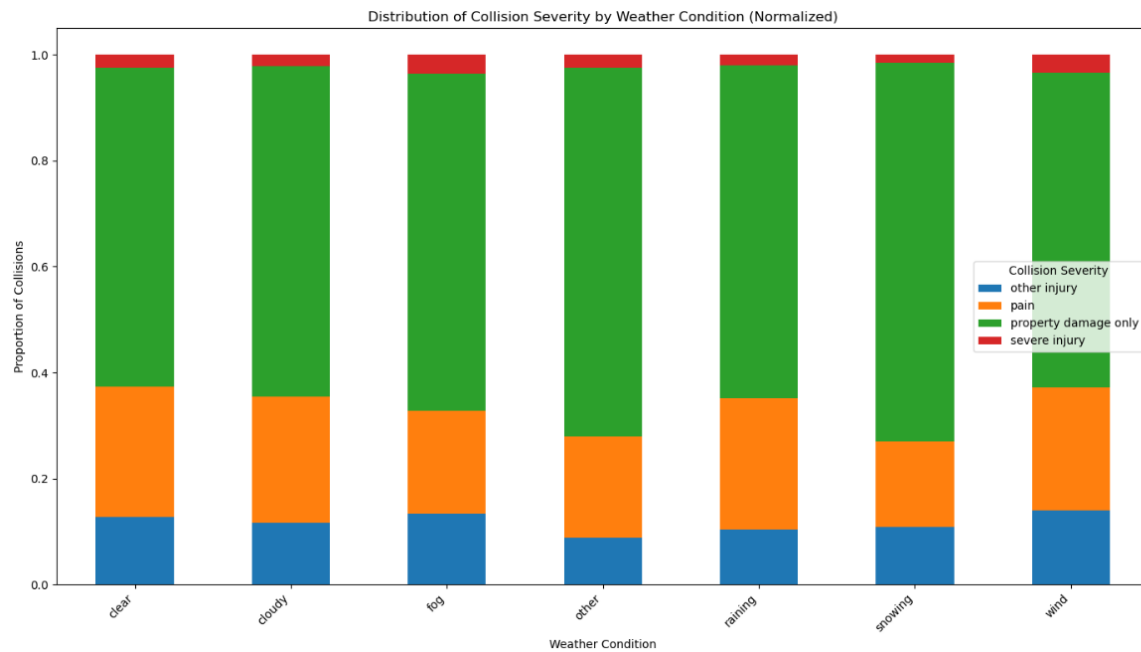
Collision Severity vs. Number of Victims

```
# Convert to Pandas for plotting
pdf = severity_victims_df.toPandas()

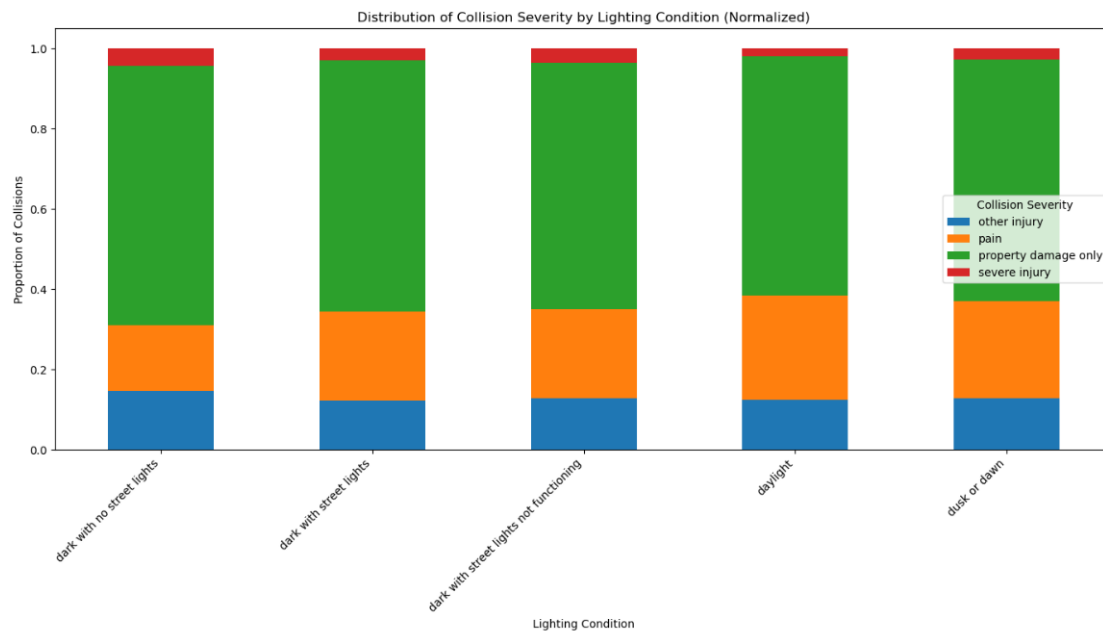
# Plot
plt.figure(figsize=(10, 6))
sns.barplot(x="collision_severity", y="avg(total_victims)", data=pdf)
plt.xlabel("Collision Severity")
plt.ylabel("Average Number of Victims")
plt.title("Collision Severity vs. Average Number of Victims")
plt.show()
```



Weather vs. Collision Severity

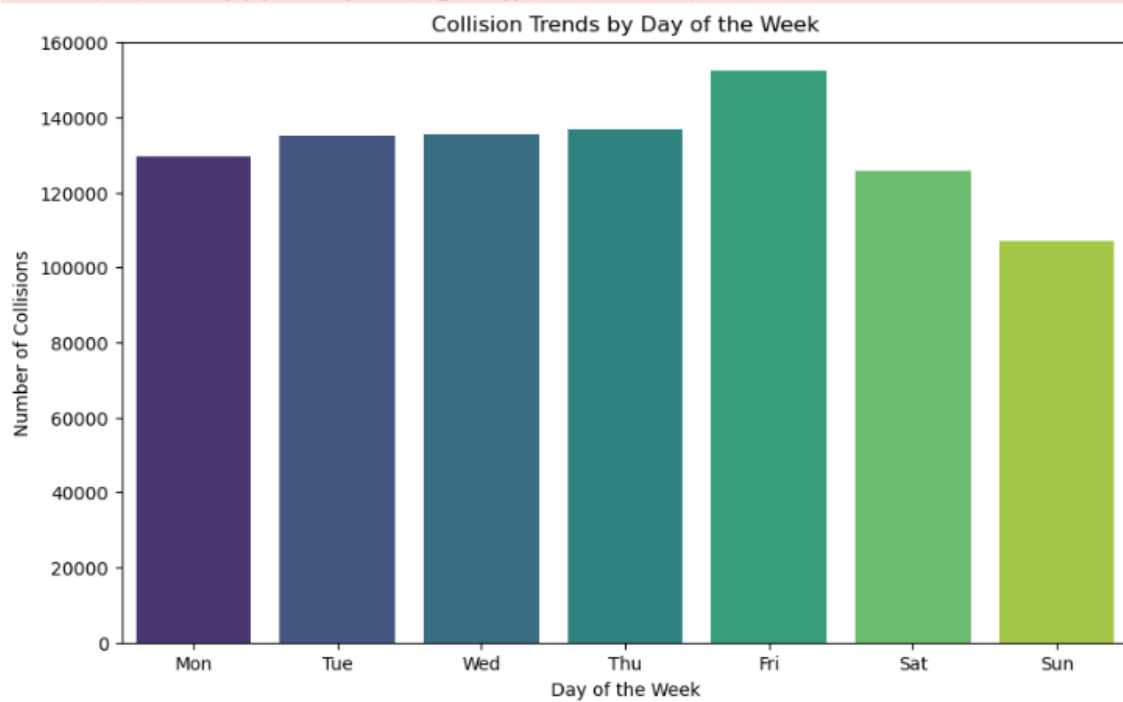


Lighting Conditions vs. Collision Severity

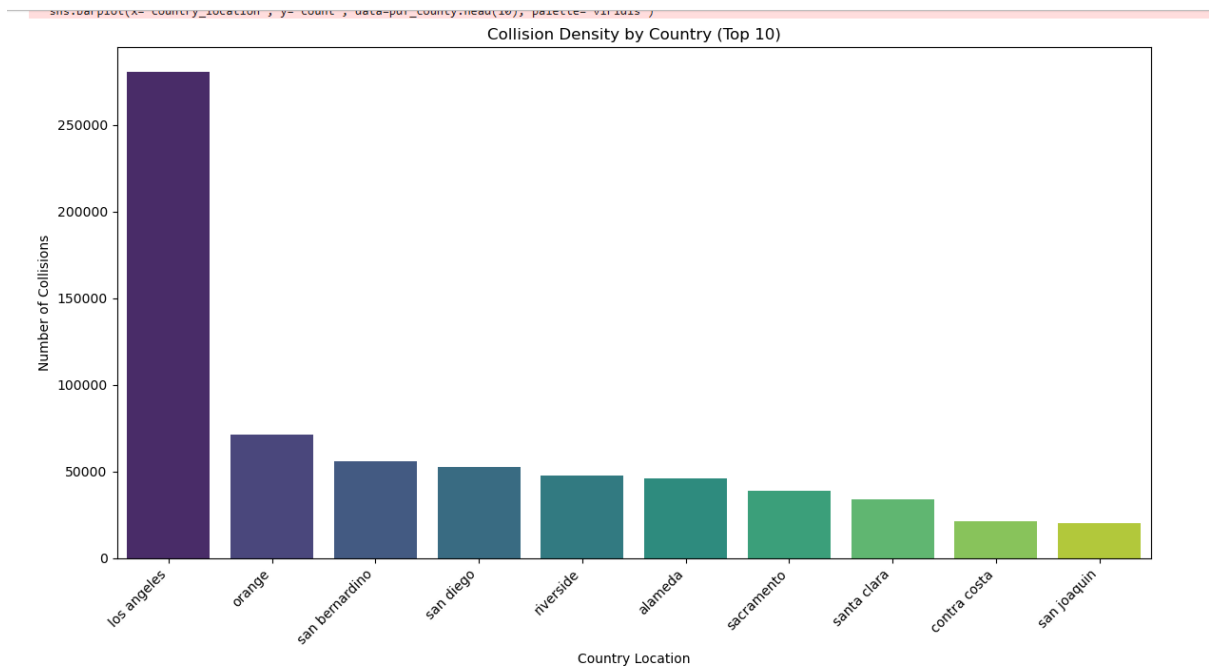


Collision count on different days. On Friday, we have more collisions.

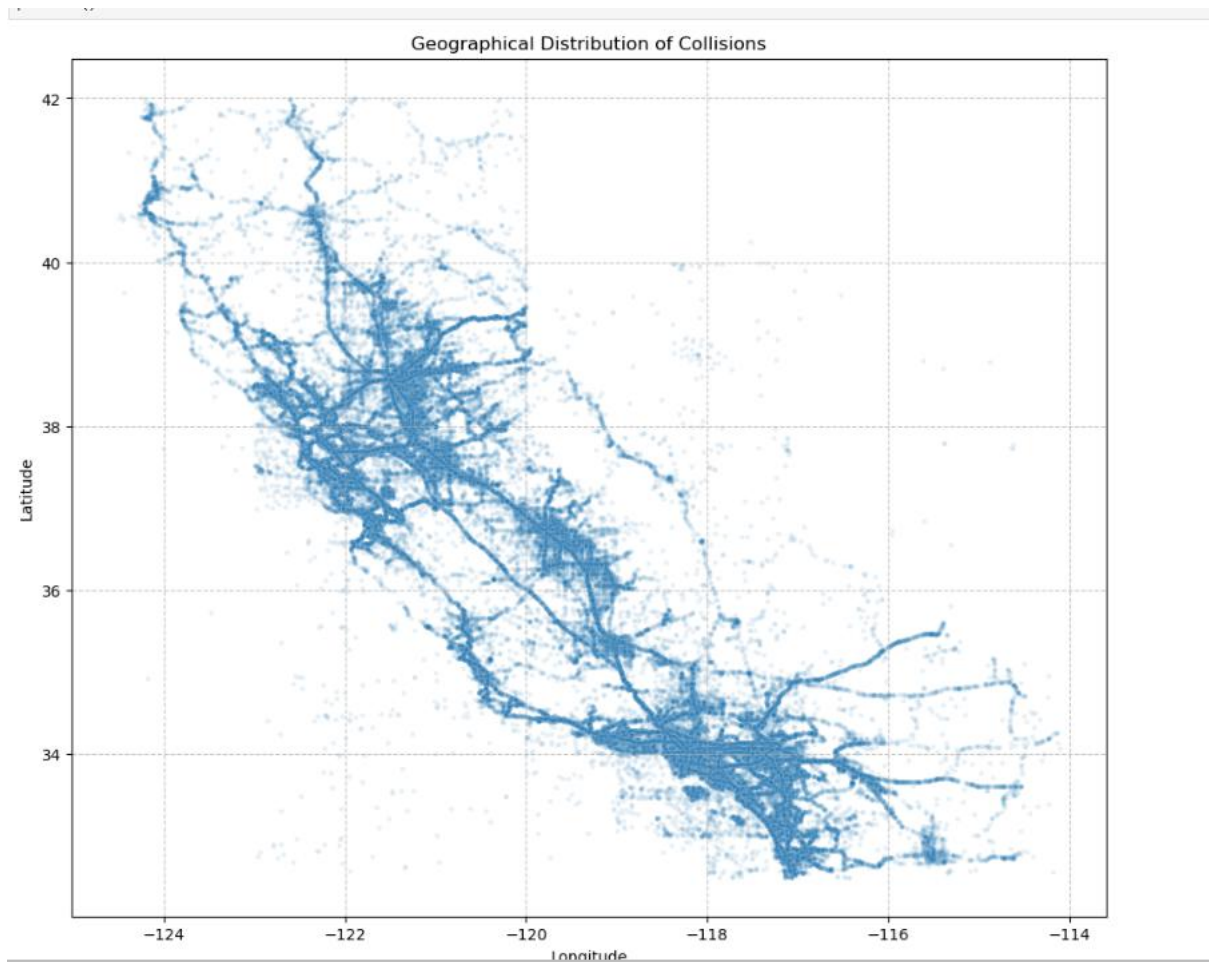
```
sns.barplot(x= weekday , y= count , data=pur_weekday, palette= viridis )
```



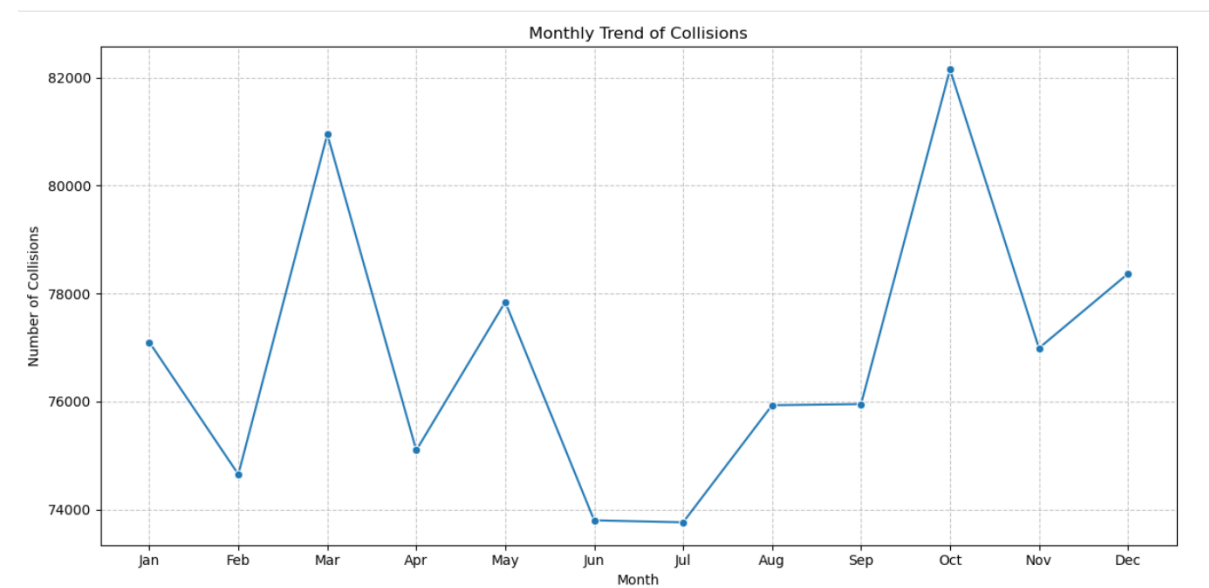
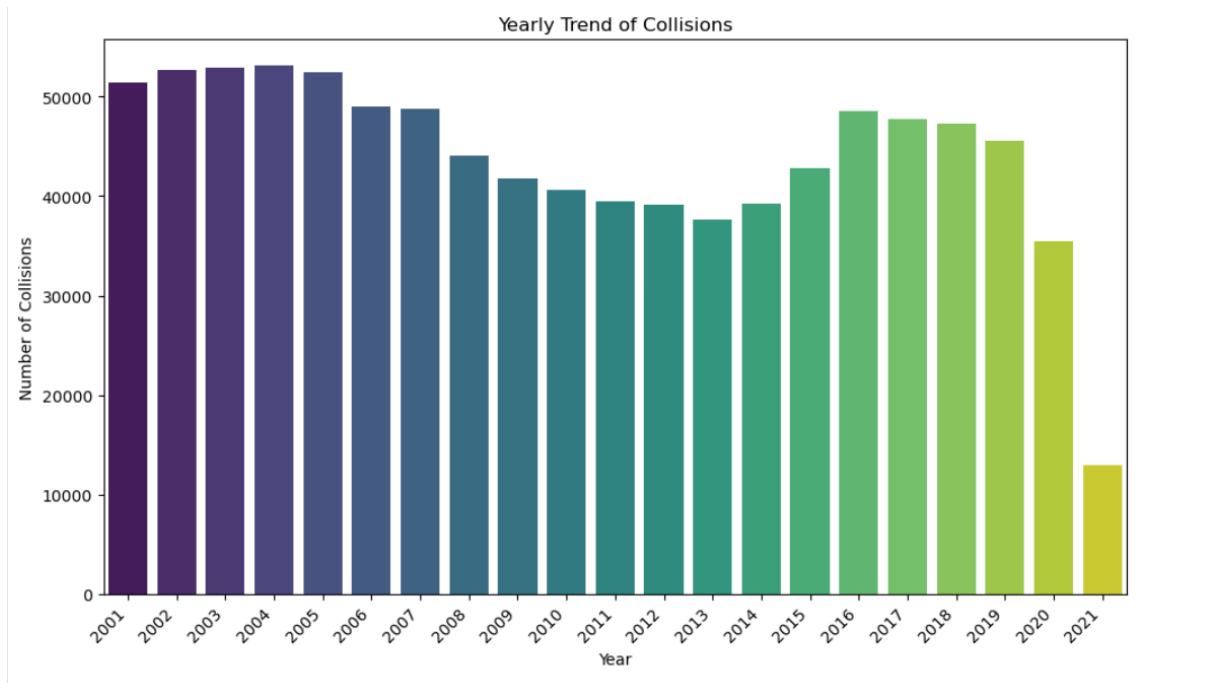
Collision Density by County



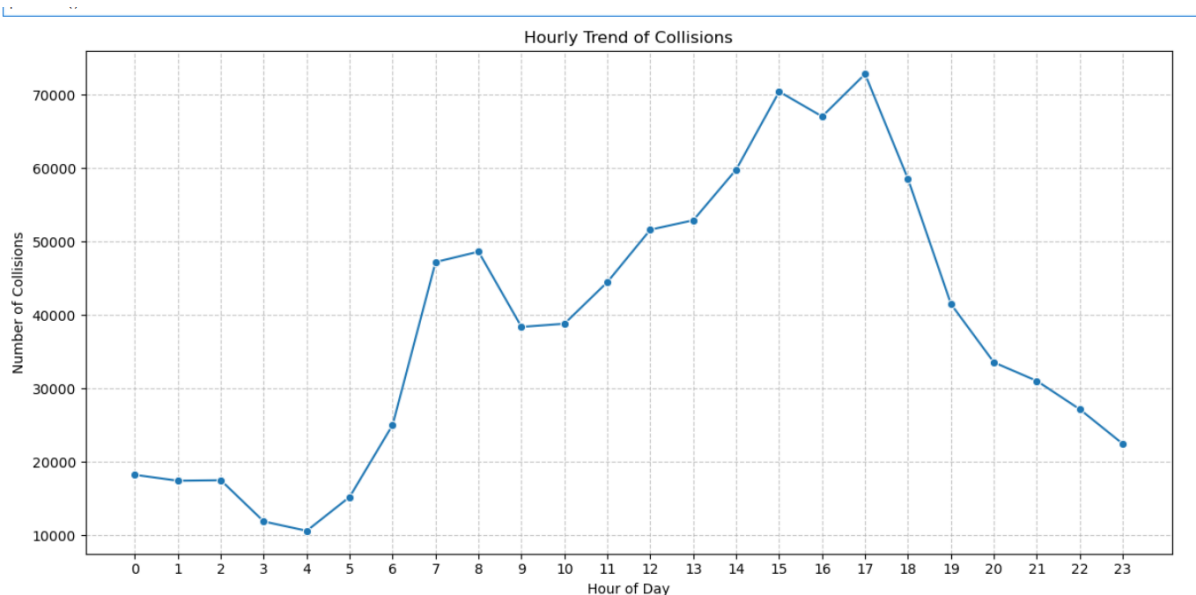
Scatter Plot of Collision Locations



Extracting the year, month and time data from collision_date and collision_time columns
And following are the year, month and hour wise trends



More collisions occurred on October month



More collisions happening in the evening.

4. ETL Querying

4.1. Top 5 Countries

```
+-----+-----+
|country_location| count|
+-----+-----+
|    los angeles|280844|
|      orange| 71136|
| san bernardino| 56176|
|    san diego| 52671|
|    riverside| 47873|
+-----+-----+
only showing top 5 rows
```

4.2. Month with Highest Collisions - OCTOBER

4.3. Weather condition with Highest collision – CLEAR

4.4. The percentage of collisions that resulted in fatalities is 0%

4.5. The most dangerous time of day for collisions is 5 PM

4.6. The top 5 road types with the highest collision frequency

Top 5 road surfaces with the highest collision frequency:

```
+-----+-----+
|road_surface|count |
+-----+-----+
|dry         |841716|
|wet         |75760 |
|snowy       |4067  |
|slippery    |1025  |
|H           |10    |
+-----+-----+
```

4.7. The top 3 lighting conditions that lead to the most collisions


```
# Query: Find the top 3 lighting conditions that lead to the most collisions
collision.groupBy('lighting').count().orderBy(desc('count')).show(3, truncate=False)
```

```
+-----+-----+
|lighting|count|
+-----+-----+
|daylight|624053|
|dark with street lights|191895|
|dark with no street lights|72774|
+-----+-----+
only showing top 3 rows
```

Final Insights and Recommendations:

This analysis of California traffic collision data has revealed several key insights that can inform road safety initiatives and urban planning. By leveraging PySpark, we were able to process and analyse large datasets to identify patterns and critical factors contributing to traffic incidents.

Key Findings:

1. **High-Risk Locations:** The counties of Los Angeles, Orange, San Bernardino, San Diego, and Riverside consistently recorded the highest number of collisions. These areas represent significant high-risk zones requiring targeted interventions.
2. **Peak Accident Times:** Collisions show distinct temporal trends, with October identified as the month with the highest number of incidents and 5 PM (17:00) as the most dangerous time of day. This suggests a correlation with rush hour traffic and potentially reduced visibility during twilight hours.
3. **Environmental Factors:** While 'clear' weather was the most common condition during collisions, 'dry' road surfaces also accounted for the majority of incidents. This highlights that human factors and infrastructure design might play a more significant role than adverse weather in many collisions. However, other conditions like 'wet' and 'snowy' surfaces also contribute to a notable number of accidents, necessitating appropriate road maintenance.
4. **Lighting Conditions:** Further analysis of lighting conditions revealed their impact on collision severity, with certain conditions (e.g., 'daylight', 'dark with streetlights', 'dark without streetlights') being more prevalent. This emphasizes the importance of adequate street lighting, especially during darker hours.
5. **Collision Severity:** The distribution of collision severity and its relationship with the number of victims indicates that even minor collisions can result in injuries, underscoring the need for preventative measures across all accident types.

Recommendations:

1. **Targeted Infrastructure Improvements:** Focus on high-collision counties (Los Angeles, Orange, San Bernardino, San Diego, Riverside) for infrastructure enhancements. This includes improving road design, signage, and intersection layouts. Implement traffic calming measures and improve pedestrian/cyclist infrastructure in these areas.
2. **Optimized Traffic Management:** Adjust traffic signal timings and implement variable speed limits during peak hours, particularly around 5 PM, to manage congestion and reduce collision risk. Public awareness campaigns could also be launched to caution drivers during high-risk months like October.

3. **Data-Driven Policy Changes:** Develop policies that address common collision factors even in clear weather and dry road conditions, emphasizing driver behaviour (e.g., distracted driving, speeding). For pedestrian and cyclist safety, policies should focus on creating dedicated lanes, improving crosswalk visibility, and stricter enforcement of traffic laws around vulnerable road users.
4. **Proactive Intervention in High-Risk Zones:** Utilize geographic collision density data to identify micro-hotspots within the high-risk counties. Deploy additional traffic patrols during peak times and implement temporary safety measures (e.g., increased lighting, warning signs) in these specific areas.
5. **Environmental Factor Mitigation:** While dry roads are prevalent in collisions, invest in improved drainage systems and road surfacing materials to minimize hazards during wet or icy conditions. Conduct regular maintenance checks for road surface integrity.
6. **Predictive Modelling:** Develop predictive models using historical data to anticipate future collision hotspots and peak times. This would allow authorities to proactively allocate resources, deploy preventative measures, and conduct targeted enforcement, moving from reactive to proactive safety management.

By implementing these data-driven recommendations, transportation authorities can significantly enhance road safety, reduce accident rates, and improve the overall efficiency of traffic management across California.